

Python fundamentals for data work

COMPLETE LEARNING GUIDE

What You'll Learn

Every data science workflow — from a quick exploratory notebook to a production ML pipeline — is built on a small set of Python fundamentals used constantly and in slightly different ways than typical application code. This guide walks through those fundamentals specifically through the lens of data work: how variables and types behave when they're holding thousands of rows instead of one value, how the built-in data structures (lists, dicts, tuples, sets) map onto real datasets, how functions and comprehensions let you transform data concisely, and the handful of conventions (naming, vectorized thinking, avoiding explicit loops) that separate code that works from code that scales. By the end, you should be able to read and write Python comfortable enough that the data libraries covered later (NumPy, Pandas, scikit-learn) feel like a natural extension rather than a new language.

1. Variables, Types, and Why Types Matter More in Data Work

Python is dynamically typed — a variable's type is determined by the value it holds, and can change. In application code this rarely causes problems. In data work it causes a huge fraction of real bugs: a column that looks numeric but is actually stored as text (because one row had a stray space or a currency symbol), a date read in as a string, a missing value silently becoming the string 'NaN' instead of an actual null. The core numeric/text/boolean types (int, float, str, bool) are the same ones you already know, but in data work you constantly interrogate them explicitly rather than trusting them: checking `type()` and `dtype` on data as a first debugging step is one of the single highest-value habits to build early.

- int and float behave differently in division: `7 / 2` is 3.5 (true division), but `7 // 2` is 3 (floor division) — mixing these up silently produces wrong aggregate statistics.
- None represents 'no value' in plain Python; pandas/numpy have their own missing-value markers (NaN, NaT, pd.NA) that behave differently in comparisons — `None == None` is True, but `NaN == NaN` is False.
- `str.strip()`, `str.lower()`, and explicit type casting (`int()`, `float()`, `str()`) are the three tools you'll reach for constantly when cleaning messy real-world data.

2. Lists, Tuples, Dicts, Sets — Choosing the Right One

These four built-in containers cover almost every data-shape you'll need before reaching for a dedicated data library. A list is an ordered, mutable sequence — the natural fit for a column of values or a sequence of rows. A tuple is ordered and immutable — used for fixed-size groupings like (row_id, value) pairs, or as dictionary keys when a list wouldn't be hashable. A dict maps keys to values and is the natural representation for a single row of structured data (e.g. {'name': 'Aisha', 'score': 88}) or for building a lookup table. A set stores unique, unordered values and is the fastest way to check membership or de-duplicate — checking `x in my\_set` is $O(1)$ on average versus $O(n)$ for a list.

PYTHON

```
students = [
    {"name": "Aisha", "score": 88},
    {"name": "Rohan", "score": 72},
    {"name": "Meera", "score": 95},
]

# Unique names seen so far (a set – fast membership checks)
seen_names = {s["name"] for s in students}

# A lookup table from name -> score (a dict comprehension)
score_by_name = {s["name"]: s["score"] for s in students}

print(score_by_name["Meera"]) # 95
print("Rohan" in seen_names) # True
```

3. Loops, Comprehensions, and 'Vectorized Thinking'

A standard for-loop over a list works, but data science code is judged partly on whether it thinks in comprehensions and, later, in vectorized operations rather than explicit Python-level loops. A list comprehension expresses 'build a new list by transforming/filtering an existing one' in a single readable line, and is meaningfully faster than the equivalent explicit loop because it avoids repeated attribute lookups Python's loop machinery pays for. This same 'operate on the whole collection at once' instinct is exactly what you'll apply at a much larger scale once NumPy/Pandas arrays replace plain lists — the mental model you build now with comprehensions is the same one that makes vectorized NumPy code make sense later.

- `[x**2 for x in range(10)]` builds a list of squares in one line — no manual `.append()` calls.
- `[x for x in scores if x >= 60]` is a filtered comprehension — the equivalent of a WHERE clause on a plain list.
- A dict comprehension (`{k: v for k, v in ...}`) and set comprehension (`{x for x in ...}`) follow the exact same pattern with different brackets.
- Deeply nested comprehensions (more than 2 levels) hurt readability more than they help — that's the point where a named function or explicit loop becomes the better choice.

4. Functions as the Unit of Reusable Data Logic

Any data-cleaning or transformation step you perform more than once belongs in a function, not copy-pasted. Python functions support default argument values (useful for optional parameters like a threshold or a column name), `*args/**kwargs` for flexible signatures, and — critically for data work — can be passed as values to other functions. This last property is what makes `.apply()`, `.map()`, and sorting with a custom key function possible: a function is just another object you can hand to pandas or the built-in `sorted()` to say 'use this logic, row by row.'

PYTHON

```
def normalize_score(raw_score: float, max_possible: float = 100) -> float:
    """Convert any score to a 0-1 scale."""
    return raw_score / max_possible

scores = [88, 72, 95, 60]
normalized = [normalize_score(s) for s in scores]
print(normalized) # [0.88, 0.72, 0.95, 0.6]

# The same function, handed to sorted() as the sort key
top_to_bottom = sorted(scores, key=normalize_score, reverse=True)
```

5. String Handling for Messy Real-World Data

Real datasets are rarely clean. A huge share of practical data science work is string cleaning: trimming whitespace, standardizing casing, splitting a combined field ('City, State') into separate columns, and matching patterns to extract structured pieces from unstructured text. Python's str methods

(`.strip()`, `.split()`, `.replace()`, `.lower()`/`.upper()`, `.startswith()`/`.endswith()`) handle the majority of cases; the re module (regular expressions) handles anything pattern-based, like extracting all digits from a phone number field or validating an email format.

- 'Bangalore, KA'.strip().split(',') gives ['Bangalore', 'KA'] — trim first, then split.
- 'CustomerID_00231'.split('_')[-1] pulls the trailing numeric part off a compound ID field.
- `re.sub(r'[^\d]', '', phone)` strips everything except digits from a messy phone number field.
- Always `.lower()` before comparing or grouping text — 'Bangalore' and 'bangalore' should usually be treated as the same category.

6. File I/O: Reading and Writing Data

Before touching pandas, it's worth understanding plain file I/O, because it's what pandas' `read_csv/read_json` are doing under the hood, and you'll occasionally need it directly for formats pandas doesn't handle well. Python's built-in `open()` supports text mode ('r', 'w', 'a') and should almost always be used with a ``with`` block, which guarantees the file is closed even if an error occurs partway through — a resource-leak bug that's easy to introduce without it. The csv and json standard-library modules parse the two most common plain-text data formats without needing any external library at all.

PYTHON

```
import csv

with open("students.csv", newline="") as f:
    reader = csv.DictReader(f)
    rows = [row for row in reader]

print(rows[0]["name"]) # each row is already a dict, keyed by header

# Writing is the mirror image
with open("summary.csv", "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=["name", "score"])
    writer.writeheader()
    writer.writerows(rows)
```

7. Scripting for Automation — Turning One-Off Code Into a Reusable Tool

A notebook cell you run once is exploration; a script you can run repeatedly, on new data, without editing it each time, is automation — and that shift is what makes a data skill valuable in a real job. Turning exploratory code into a script means: moving hard-coded values (file paths, thresholds) into named constants or command-line arguments (the `argparse` module is the standard tool for this), wrapping the logic in functions with clear inputs/outputs instead of top-to-bottom cell execution, and adding basic error handling for the things that will eventually go wrong (a missing file, an empty dataset, an unexpected column).

- if `__name__ == '__main__':` is the standard guard that lets a file be both imported as a module and run directly as a script.
- `argparse.ArgumentParser()` turns 'change this value and re-run' into 'python `clean_data.py --input raw.csv --threshold 0.8`' — no code editing required.
- Logging (the logging module) instead of scattered `print()` statements is what separates a script you can debug in production from one you can only debug by re-running it.

Common Pitfalls

- Assuming a column read from a file is numeric just because it looks numeric — always check `.dtype` or `type()` before doing arithmetic on it.
- Comparing floating-point numbers with `==` (`0.1 + 0.2 == 0.3` is actually `False` due to floating-point representation) — use a small tolerance comparison instead.
- Forgetting that strings are compared and sorted lexicographically, not numerically — `'10' < '9'` is `True` as strings.
- Mutating a list while iterating over it in a `for` loop, which silently skips elements or raises confusing errors.

- Using a mutable default argument (`def f(items=[])`) — the same list object is reused across every call, quietly accumulating state between unrelated calls.

Key Takeaways

- Type-check messy real data explicitly rather than assuming it's clean — this single habit prevents the largest category of data bugs.
- Comprehensions aren't just shorter syntax — they build the 'operate on the whole collection' mindset you'll need for vectorized NumPy/Pandas code.
- Every container (list/tuple/dict/set) has a specific job; picking the right one is a design decision, not a style preference.
- Always open files with a `with` block — it's the difference between code that reliably closes resources and code that occasionally leaks them.
- The path from 'a notebook cell that worked once' to 'a script someone else can run on new data' runs through functions, named constants, and basic error handling.

Learn More

Python Official Documentation — Data Structures

<https://docs.python.org/3/tutorial/datastructures.html>

Real Python — Python Data Types

<https://realpython.com/python-data-types/>

GeeksforGeeks — Python String Methods

<https://www.geeksforgeeks.org/python-string-methods/>